

OCR Pipeline in DSML

1. Introduction

An OCR (Optical Character Recognition) pipeline is a system that extracts text from images or scanned documents and converts it into machine-readable structured data.

2. Steps in OCR Pipeline

2.1 Image Preprocessing

Improves image quality using grayscale conversion, noise removal, and thresholding.

```
import cv2
image = cv2.imread("invoice.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)
cv2.imwrite("processed.jpg", thresh)
```

2.2 Text Detection

Detects text regions using contours or deep learning models.

```
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(image, (x,y), (x+w, y+h), (0,255,0), 2)
```

2.3 Text Recognition

Extracts text using OCR engines like Tesseract or EasyOCR.

```
import pytesseract
from PIL import Image
text = pytesseract.image_to_string(Image.open("processed.jpg"))
print(text)
```

2.4 Post-processing

Cleans extracted text using regex or NLP.

```
import re
text = "Invoice No: 12345 Amount: Rs.5000"
invoice_no = re.search(r'\d+', text).group()
amount = re.search(r'Rs\.\d+', text).group()
```

2.5 Structured Output

Converts extracted data into JSON format.

```
data = {"invoice_no": invoice_no, "amount": amount}
print(data)
```

3. Full Pipeline Example

```
import cv2
import pytesseract
from PIL import Image

image = cv2.imread("invoice.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY)
cv2.imwrite("temp.jpg", thresh)

text = pytesseract.image_to_string(Image.open("temp.jpg"))
print(text)
```

4. Applications

Used in invoice processing, KYC verification, document digitization, and NLP-based analytics.